

# rdd\_quickup

February 19, 2026

```
[1]: from pyspark.sql import SparkSession

spark=SparkSession.builder.appName("revision").getOrCreate()

sc=spark.sparkContext
```

```
[ ]: # partitioning
rdd.getNumPartitions()

# set partitions
sc.parallelize(data,4)

# repartition
rdd.repartition(4)
rdd.coalesce(1)
```

```
[8]: # creation from collcection
l=[2,3,4,5]
rdd1=sc.parallelize(l)

# from external file
rdd=sc.textFile("prod.csv")
rdd.collect()
# rdd.take(10)
# header
header= rdd.first()
```

```
[7]: # Transformations --> map,filter,flatMap,distinct,union,join,reduceByKey
# actions(execute the dag)-->collect,count,first,take,reduce,saveAsTextFile
```

```
[9]: # remove header
no_h=rdd.filter(lambda x:x!=header)
```

```
[11]: # Map
# one input --> one output
r1=sc.parallelize(["This is random String"])
r1.map(lambda x:x.split(" ")).collect()
```

```
[11]: [['This', 'is', 'random', 'String']]
```

```
[13]: # FlatMap() --> one input --> multiple outputs(flattened)
r1.flatMap(lambda x:x.split(" ")).collect()
```

```
[13]: ['This', 'is', 'random', 'String']
```

```
[14]: # filter -->filter based on condition
```

```
[17]: # Key - Value RDD
# (key,value)
rdd = sc.parallelize(["a 1", "b 2", "a 3"])
pairs = rdd.map(lambda x: (x.split()[0], int(x.split()[1])))
pairs.collect()
```

```
[17]: [('a', 1), ('b', 2), ('a', 3)]
```

```
[18]: # reduceByKey()
# efficient aggregation
pairs.reduceByKey(lambda x,y:x+y).collect()
```

```
[18]: [('a', 4), ('b', 2)]
```

```
[19]: # groupByKey --> slower
pairs.groupByKey().mapValues(sum).collect()
```

```
[19]: [('a', 4), ('b', 2)]
```

```
[ ]: # word count example
r1=sc.textFile("sample.txt")
wordcount=(
    r1.flatMap(lambda x:x.split(" "))
    .map(lambda x:(x,1)).reduceByKey(lambda x,y:x+y)
)
```

In RDD, “group by column” = Convert column into KEY → then groupByKey/reduceByKey.

```
[ ]: # Example , group by city
pair_dd=rdd.filter(lambda x:x!=header).map(lambda x:x.split(",")).map(lambda x:
    ↪(x[2],[3]))
#(city,name)
# now
pair_rdd.groupByKey().collect()

# or
pair_rdd.mapValues(lambda x:list(x)).collect()
```

```
[ ]: # distinct
rdd.distinct() # removes duplicates

# union
rdd1.union(rdd2)

# intersection
# common elements
```

```
[ ]: # sorting
# sortByKey()
pairs.sortByKey().collect()

# sortBy
rdd.sortBy(lambda x:x[1],ascending=False) # descending
```

```
[ ]: # Join
rdd1 = sc.parallelize([(1, "Tarun"), (2, "Ali")])
rdd2 = sc.parallelize([(1, "Premium"), (2, "Regular")])

rdd1.join(rdd2).collect()
```

```
[ ]: # Narrow Transformations (no shuffle) --> map,filter,flatMap
# wide (shuffle happens)--> reduceByKey,groupByKey,join,distinct
```

```
[ ]: # proper template
rdd = sc.textFile("file.txt")

header = rdd.first()

cleaned = (
    rdd.filter(lambda x: x != header)
    .map(lambda x: x.split(","))
    .filter(lambda x: len(x) == expected_columns)
)
```

Full Example

```
[ ]: # Full qsn
# For each city, calculate:
# Total sales
# Total number of orders
# Average order value
# Only keep cities where total sales > 700

rdd=sc.textFile("orders.txt")
h=rdd.first()
```

```

cleaned=rdd.filter(lambda x:x!=h).map(lambda x:x.split(","))
cleaned.collect()

# (key,value)
# key =city, value=(amount,1)
pair=cleaned.map(lambda x:(x[2],(float(x[4]),1))) # this is for better shuffle,
↳we can do it separately also(but this takes multiple shuffles)

# aggregations
# sum of amount(total sales)
# total number of orders
aggregated=pair.reduceByKey(lambda x,y:(x[0]+y[0],x[1]+y[1])) # (total
↳sales,total orders)

# avg
final=aggregated.mapValues(lambda x:(x[0],x[1],x[0]/x[1]))

# filter total sales>700
res=final.filter(lambda x:x[1][0]>700)
# sort by total sales descending
res1=res.sortBy(lambda x:x[1][0],ascending=False)

#print
for city,values in res1.collect():
    print(f"{city},{values[0]},{values[1]},{values[2]}")

```

Houston,1000.0,3,333.3333333333333

```

[ ]: # max order value per city
# min order value per city

pair=cleaned.map(lambda x:(x[2],float(x[4])))

max_order=pair.reduceByKey(lambda x,y:x if x>y else y) # or lambda x,y:max(x,y)

min_order = pair.reduceByKey(lambda x, y: x if x < y else y)

```

| Question Type | What To Use                      |
|---------------|----------------------------------|
| Count         | (key,1) + reduceByKey            |
| Sum           | (key,value) + reduceByKey        |
| Max           | reduceByKey(max)                 |
| Min           | reduceByKey(min)                 |
| Average       | (value,1) + reduceByKey → divide |
| Join          | rdd1.join(rdd2)                  |